

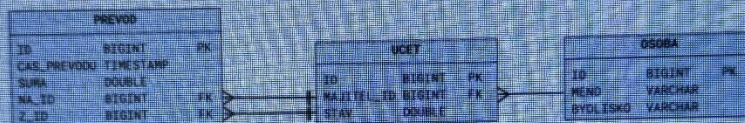
V zip súbore, ktorý ste si stiahli sa nachádzajú aj adresáre **dbapp** a **restapp**.

Implementačná úloha 1.

dbapp je maven-projekt, pre implementáciu databázovej aplikácie. Projekt už obsahuje:

- súbor **pom.xml** so všetkými závislosťami pre implementáciu zadania, nie je ho potrebné modifikovať.
 - súbor **persistence.xml** s konfiguračnými údajmi pre databázu postgres (beží vo virtuálnom stroji).
- Poznámka. Postgres je referenčná databáza pre testovanie a hodnotenie implementácie. Pri vývoji môžete použiť aj inú databázu, musíte si však upraviť **persistence.xml**.
- Balík **banka** so súbormi tried:
 - **Osoba**, **Ucet** a **Prevod** sú entitné triedy do ktorých treba doplniť JPA anotácie
 - a trieda **DbApp** obsahujúca dve metódy, ktoré treba implementovať.

A) Do entitných tried **Osoba**, **Ucet** a **Prevod** doplňte JPA anotácie, tak aby sa tieto entitné triedy namapovali na databázové tabuľky znázornené na diagrame:



Dôležité: Anotácie doplňte tak, aby z entitných tried bolo možné vygenerovať tabuľky presne takou štruktúrou, ako je uvedená v diagrame, teda:

- Názvy tabuľiek, stĺpcov musia byť zachované.
- Primárny kľúč **ID** v tabuľke **PREVOD** musí byť **autogenerovaný**.
- Čudzie kľúče **Z_ID** a **NA_ID** a stĺpec **SUMA** sú **povinné** (nie nullable)

Vysvetlenie k významu tabuľiek a stĺpcov:

- Tabuľka **OSOBA** obsahuje informácie osobách, zákazníkoch banky: **MENO**, **ADRESA** a **ID** – primárny kľúč.
 - Tabuľka **UCET** obsahuje informácie o účtoch: číslo účtu **ID** je zároveň primárny kľúč, **MAJITEL_ID** je osoba, ktorej účet patrí a **STAV** udáva aktuálnu sumu peňazí na účte.
 - Tabuľka **PREVOD** obsahuje informácie o prevodoch peňazí medzi účtami:
 - **SUMA** je prevádzaná suma,
 - **Z_ID** číslo účtu, z ktorého sa suma prevádza
 - **NA_ID** číslo účtu, na ktorý sa suma prevádza.
 - V stĺpci **CAS_PREVODU** je čas (Timestamp) realizácie prevodu.
- Ak prevod ešte nebol realizovaný nie je v ňom žiadna hodnota.

B) Súbor **DbApp.java** obsahuje deklarácie dvoch metód:

```

public static long novýPrevod(EntityManager em, double suma,
                             long zUctu, long naUcet) throws Exception
public static int realizujPrevod(EntityManager em)
  
```

Implementujte metódy podľa špecifikácie, ktorú nájdete v zdrojovom súbore **DbApp.java**, v komentári k metódam.

Poznámky k implementácii:

Odporúčam postupovať tak, že najprv dokončíte implementáciu entitných tried a z nich si necháte vygenerovať tabuľky. Nie je potrebné generovať **JPAController** a ani sa neodporúča jeho použitie.

Pre vlastné testovanie si môžete implementovať program (napr. main metódu v triede **DbApp**), nie je to však súčasťou zadania, nebude sa testovať ani hodnotiť.

Pokyny pre odovzdanie

Odovzdáva sa iba adresár balíka **banka** zazipovaný. Pre zipovanie použite formát **.zip** a súbor nazvite **banka.zip**.

Implementačná úloha 2.

restapp je maven projekt pre vývoj webovej aplikácie s REST servisom. Projekt obsahuje:

- súbor `pom.xml` so všetkými závislosťami potrebnými pre vývoj služby. Nie je ho potrebné dopĺňať.
- balík `rest`. V tomto balíku **budete vytvárať všetky triedy** (triedu implementujúcu služby aj dátové triedu). V balíku sa už nachádza konfiguračná trieda REST aplikácie `JakartaRestConfiguration`. *Netreba ju meniť.*

Špecifikácia služby

Služba bude sprístupňovať informácie o zmluvách cestovného poistenia a v nich poistených osobách.

Informácie o poistnej zmluve predstavujú zdroj, ktorého stav reprezentovaný vo formáte XML, má štruktúru:

```
<?xml version="1.0" encoding="UTF-8"?>
<poistenie>
  <idZmluvy>Z123</idZmluvy>
  <poistnaSuma>199.0</poistnaSuma>
  <majitel>
    <meno>Hrasko</meno>
    <rc>2001</rc>
    <bydlisko>Hlavna 22</bydlisko>
  </majitel>
  <pocetPoistencov>2</pocetPoistencov>
</poistenie>
```

Vysvetlenie

- element `idZmluvy` je reťazec predstavujúci jednoznačný identifikátor zmluvy.
- element `poistnaSuma` je číslo udávajúce poistnú sumu.
- element `majitel` je podelement obsahujúci údaje o majiteľovi poisťky (meno, rodné číslo a bydlisko).
- element `pocetPoistencov` je číslo udávajúce aktuálny počet osôb poistených v zmluve. Ak zmluva ešte nemá poistené osoby element obsahuje 0. Pri pridaní alebo odobratí poistenej osoby sa počet poistencov aktualizuje.

Informácie o poistených osobách:

Každá zmluva má svoj zoznam poistených osôb. XML dokument s informáciami o zmluve však obsahuje len počet poistených osôb, bližšie informácie o poistených osobách **neobsahuje**. Každá v zmluvu poistená osoba bude prístupná ako samostatný pod-zdroj zmluvy.

Koreňový zdroj služby má URL `poistenie`

Zdroj s URL `poistenie/{id}`, reprezentuje poistnú zmluvu s identifikátorom `id` (pozri element `idZmluvy`).

Zdroj s URL `poistenie/{id}/{no}`, reprezentuje osobu poistenú v zmluve. Reťazec `no` je poradové číslo v zozname zmluvou poistených osôb. Poradové čísla idú za sebou od 1 až po `pocetPoistencov`.

Inicializácia servisu:

Službu inicializujte tak, že po štarte (deploymente) bude už vytvorená jedna zmluva s nasledujúcimi údajmi:

```
idZmluvy: Z123,
poistnaSuma: 199.0
majitel: meno Hrasko, rc: 2001, bydlisko Hlavna 22
nebude mať žiadne poistené osoby, teda pocetPoistencov bude 0.
```

Prístupové metódy:

Pre koreňový zdroj `poistenie`

- **POST** pre `MediaType.APPLICATION_XML`. Služi pre vytvorenie nového zdroja s informáciami o poistnej zmluve. Akceptuje XML dokument podľa hore uvedenej špecifikácie. **Pozor však, nová zmluva nebude mať ešte žiadnych poistencov.** Teda ak by XML element `pocetPoistencov` obsahoval nenulovú hodnotu, ignoruje ju a nastaví počet poistencov zmluvy na 0. Metóda POST nič nevracia.

Pre zdroj zmluvy **poistenie/{id}**

- **GET** pre MediaType.APPLICATION_XML. Vyhľadá zmluvu podľa identifikátora zadaneho v URL a vráti informácie ako XML dokument s horeuvedenou štruktúrou. Ak zmluva neexistuje, malo by sa vrátiť HTTP 204 alebo 404. *Pomôcka: stačí vrátiť jednoducho null.*
- **POST** pre MediaType.APPLICATION_XML. Služi na pridanie poistenej osoby do zmluvy. Akceptuje XML dokument so štruktúrou:

```
<osoba>
  <meno>Mrkvicka</meno>
  <rc>20102010</rc>
  <bydlisko>Polna 1</bydlisko>
</osoba>
```

Vyhľadá zmluvu podľa jej id a pridá osobu do zoznamu poistených osôb v zmluve.

Návratová hodnota - MediaType.TEXT_PLAIN. Metóda POST vráti **poradové číslo osoby** v zozname. *Pripomínam: Číslovanie začína od 1 (teda ak vkladáte na koniec zoznamu, poradové číslo bude rovné dĺžke zoznamu).*

Ak zmluva neexistuje, metóda vráti jednoducho null.

Pre zdroj poistenej osoby **poistenie/{id}/{no}**

- **GET** pre MediaType.APPLICATION_XML. Vráti XML dokument poistenej osoby s daným poradovým číslom. Ak zmluva alebo poistená osoba neexistuje, malo by sa vrátiť HTTP 204 alebo 404. *Pomôcka: stačí vrátiť jednoducho null.*
- **DELETE** Odstráni položku zo zoznamu poistených osôb. Metóda neočakáva ani nevracia žiadne dáta. *Poznámka: Po odstránení osoby zo zoznamu poradové číslo ostatných poistených osôb v zozname môžu ale nemusia zmeniť. Nebude sa to overovať unit-testami.*

Pokyny pre implementáciu

Informácie o zmluvách (stav zdrojov) budú udržiavané len v pamäti servisu. Vytvorte si pre to vhodné dátové štruktúry a použite anotáciu **@jakarta.inject.Singleton**. Nezapudnite tiež na anotáciu **@XmlElement**.

Poznámka: Uvedený dokument je ukážka. XML dokumenty, s ktorými servis pracuje, budú mať elementy s rovnakými názvami a štruktúrou, odlišovať sa budú len textovým obsahom a počtom elementov. (poistených osôb môže byť viac ale nemusí byť žiadna. Poradie elementov nie je dôležité.)

Pokyny pre odovzdávanie

Odozdáva sa iba adresár balíka **rest** zazipovaný. Pre zipovanie použite formát **.zip**, súbor nazvite **rest.zip**.